

Phase 3 Test Specifications

POC-07 — Inventory Management & Procurement System

Total Test Cases: 20 | **Pass Threshold:** 14 of 20 (70%)

TOOL DEFINITION TESTS (4 cases)

TC-07-P3-TOOL-01: All Tools Have Names and Descriptions

```
def test_tools_defined():
    from agent.tools import (get_low_stock_alerts, get_product_stock,
                             get_supplier_catalog, get_dashboard_stats,
                             search_inventory_policy)
    for t in [get_low_stock_alerts, get_product_stock, get_supplier_catalog,
              get_dashboard_stats, search_inventory_policy]:
        assert t.name and t.description
```

TC-07-P3-TOOL-02: All 5 Tools Registered in Executor

```
def test_tools_registered():
    from agent.agent import build_agent_executor
    names = [t.name for t in build_agent_executor().tools]
    for r in ["get_low_stock_alerts", "get_product_stock", "get_supplier_catalog",
              "get_dashboard_stats", "search_inventory_policy"]:
        assert r in names
```

TC-07-P3-TOOL-03: Descriptions At Least 50 Characters

```
def test_desc_length():
    from agent.tools import (get_low_stock_alerts, get_product_stock,
                             get_supplier_catalog, get_dashboard_stats,
                             search_inventory_policy)
    for t in [get_low_stock_alerts, get_product_stock, get_supplier_catalog,
              get_dashboard_stats, search_inventory_policy]:
        assert len(t.description) >= 50, f"{t.name} description too short"
```

TC-07-P3-TOOL-04: Tools Accept String Input

```
def test_string_input():
    from agent.tools import get_product_stock, get_supplier_catalog
    for t in [get_product_stock, get_supplier_catalog]:
```

```
if t.args_schema:
    assert len(t.args_schema.schema().get("properties", {})) >= 1
```

EXECUTION TESTS (6 cases)

TC-07-P3-EXEC-01: Dashboard Returns Inventory Metrics

```
def test_dashboard_counts():
    from unittest.mock import patch
    mock = {"total_products": 500, "low_stock_count": 25, "out_of_stock_count": 5,
            "open_po_count": 8, "total_stock_value": 2500000.0}
    with patch("agent.tools._api_get", return_value=mock):
        from agent.tools import get_dashboard_stats
        result = get_dashboard_stats.invoke("")
        assert "500" in result or "product" in result.lower()
        assert "25" in result or "low" in result.lower()
```

TC-07-P3-EXEC-02: Product Stock Returns Quantities

```
def test_product_stock():
    from unittest.mock import patch
    mock = {"id": 1, "sku": "SKU-GRO-0001", "name": "Basmati Rice",
            "reorder_point": 20, "reorder_quantity": 100,
            "stock_level": {"quantity_on_hand": 15, "quantity_available": 15,
            "quantity_reserved": 0}}
    with patch("agent.tools._api_get", return_value=mock):
        from agent.tools import get_product_stock
        result = get_product_stock.invoke("1")
        assert "15" in result or "available" in result.lower()
```

TC-07-P3-EXEC-03: 404 Product Returns Not Found

```
def test_product_404():
    from unittest.mock import patch
    with patch("agent.tools._api_get", return_value={"error": "not_found"}):
        from agent.tools import get_product_stock
        result = get_product_stock.invoke("9999")
        assert "not found" in result.lower() or "9999" in result
```

TC-07-P3-EXEC-04: Policy Search Returns PO Rules

```
def test_policy_search():
    from unittest.mock import patch
```

```
mock = {"answer": "PO approval required for orders above ₹50,000."}
with patch("rag.rag_chain.build_rag_chain"), \
    patch("rag.rag_chain.ask_question", return_value=mock):
    from agent.tools import search_inventory_policy
    result = search_inventory_policy.invoke("When is PO approval required?")
    assert "50,000" in result or "approval" in result.lower()
```

TC-07-P3-EXEC-05: Low Stock Returns Alert List

```
def test_low_stock():
    from unittest.mock import patch
    mock = [{"id": 1, "sku": "SKU-GRO-0001", "name": "Rice", "quantity_available":
5, "reorder_point": 20}]
    with patch("agent.tools._api_get", return_value=mock):
        from agent.tools import get_low_stock_alerts
        result = get_low_stock_alerts.invoke("")
        assert "SKU-GRO" in result or "reorder" in result.lower() or "1" in result
```

TC-07-P3-EXEC-06: API Unavailable Returns Error Dict

```
def test_api_unavailable():
    from unittest.mock import patch
    from agent.tools import _api_get
    with patch("agent.tools.requests.get",
side_effect=ConnectionError("refused")):
        result = _api_get("/products/1")
        assert isinstance(result, dict) and "error" in result
```

CONTEXT MANAGEMENT TESTS (4 cases)

TC-07-P3-CTX-01: System Prompt Has Inventory Role

```
def test_system_prompt_role():
    from agent.prompts import INVENTORY_AGENT_SYSTEM_PROMPT
    assert any(w in INVENTORY_AGENT_SYSTEM_PROMPT.lower()
        for w in ["inventory", "stock", "procurement", "retail",
"reorder"])
    assert len(INVENTORY_AGENT_SYSTEM_PROMPT) >= 100
```

TC-07-P3-CTX-02: Long Responses Summarized

```
def test_long_summarized():
    from unittest.mock import patch, MagicMock
```

```
from agent.summarizer import _summarize_if_long
long = "Stock report: " + ("SKU-GRO-0001 has 5 units. " * 100)
assert len(long) > 2000
with patch("agent.summarizer.load_summarize_chain") as mc:
    mc.return_value.run.return_value = "Summary: 100 products need reorder."
    assert isinstance(_summarize_if_long(long), str)
```

TC-07-P3-CTX-03: Short Unchanged

```
def test_short_unchanged():
    from agent.summarizer import _summarize_if_long
    short = "SKU-GRO-0001 Basmati Rice: 15 available, reorder point 20."
    assert _summarize_if_long(short) == short
```

TC-07-P3-CTX-04: System Prompt References All 5 Tools

```
def test_all_tools_referenced():
    from agent.prompts import INVENTORY_AGENT_SYSTEM_PROMPT
    count = sum(1 for t in ["get_low_stock_alerts", "get_product_stock",
                           "get_supplier_catalog", "get_dashboard_stats",
                           "search_inventory_policy"]
                if t in INVENTORY_AGENT_SYSTEM_PROMPT)
    assert count >= 4
```

END-TO-END TESTS (6 cases)

TC-07-P3-E2E-01: Low Stock Query Uses get_low_stock_alerts

```
def test_low_stock_query():
    from agent.agent import build_agent_executor
    names = [t.name for t in build_agent_executor().tools]
    assert "get_low_stock_alerts" in names
```

TC-07-P3-E2E-02: Multi-Tool Supported

```
def test_multi_tool():
    from agent.agent import build_agent_executor
    names = [t.name for t in build_agent_executor().tools]
    assert "get_product_stock" in names and "search_inventory_policy" in names
```

TC-07-P3-E2E-03: Policy RAG Query

```
def test_policy_rag():
    from unittest.mock import patch
    mock = {"answer": "Low stock alert triggers when quantity_available ≤ reorder_point."}
    with patch("rag.rag_chain.ask_question", return_value=mock):
        from agent.tools import search_inventory_policy
        result = search_inventory_policy.invoke("When is a low stock alert triggered?")
        assert "low" in result.lower() or "reorder" in result.lower()
```

TC-07-P3-E2E-04: Dashboard Query

```
def test_dashboard_query():
    from unittest.mock import patch
    mock = {"total_products": 500, "low_stock_count": 25, "out_of_stock_count": 5,
            "open_po_count": 8, "total_stock_value": 2500000.0}
    with patch("agent.tools._api_get", return_value=mock):
        from agent.tools import get_dashboard_stats
        result = get_dashboard_stats.invoke("")
        assert "500" in result or "product" in result.lower()
```

TC-07-P3-E2E-05: Ambiguous No Crash

```
def test_ambiguous_no_crash():
    from unittest.mock import patch
    with patch("agent.tools._api_get", return_value=[]):
        from agent.tools import get_low_stock_alerts
        try:
            result = get_low_stock_alerts.invoke("")
            assert isinstance(result, str)
        except Exception as e:
            pytest.fail(f"Raised: {e}")
```

TC-07-P3-E2E-06: LangSmith Trace

```
def test_langsmith_trace():
    import os
    if not os.getenv("LANGCHAIN_API_KEY"):
        pytest.skip("No key")
    from langsmith import Client
    runs = list(Client().list_runs(project_name="AI-Readiness-POC-07-P3",
limit=3))
    if not runs:
        pytest.skip("No traces yet")
    assert len(runs) >= 1
```